

System Architecture & Technical Documentation

Comprehensive Technical Reference

Offline RAG-Powered Automotive Quality Analytics & Diagnostics Platform

Document Version: 1.1

Date: June 11, 2026

Classification: **CONFIDENTIAL — INTERNAL**

Prepared by: Senior Solutions Architect

Technology Stack: Python · Streamlit · FAISS · Phi-3 · SQLite

Contents

1	System Overview	4
1.1	Core Objectives	4
1.2	System Specifications	5
2	End-to-End Architecture	6
2.1	Layered Architecture Diagram	6
2.2	Module Directory Structure	6
3	Data Flow & Processing Pipeline	8
3.1	ETL Ingestion Pipeline	8
3.2	ETL Processing Steps	8
3.3	Query Processing Data Flow	9
4	Component Interactions	10
4.1	Singleton Resource Management	10
5	Backend Workflow	11
5.1	Application Startup Sequence	11
5.2	Runtime Query Processing Steps	12
6	Database Architecture	13
6.1	Entity Relationship Diagram	13
6.2	Core Schema — <code>records</code> Table	13
6.3	Materialized Views	14
6.4	Index Strategy	15
7	API & Query Flow	16
7.1	Internal Query API Contract	16
7.2	Intent Classification Rules	16
8	AI / ML Pipeline	17
8.1	NLP Processing Pipeline	17
8.2	Named Entity Recognition	18
8.3	Local LLM Inference Engine	18
8.4	LLM-Generated SQL with Self-Healing	18
9	Retrieval Pipeline (SQL + FAISS + RAG)	20
9.1	Three-Stage Hybrid Retrieval	20
9.2	Vector Index Architecture	21
9.3	Embedding Cache Strategy	21
9.4	SQL Keyword Search (BM25-like)	21
10	Frontend Architecture	22
10.1	Page Structure	22
10.2	UI Component Hierarchy	22
10.3	Visualization Chart Types	23

- 11 Deployment Architecture** **24**
 - 11.1 System Requirements 24
 - 11.2 Startup Commands 24
- 12 Scalability Considerations** **26**
- 13 Performance Optimizations** **27**
- 14 Security Considerations** **28**
- 15 Error Handling & Resilience** **29**
- 16 Future Improvements** **30**
 - 16.1 Short-Term Enhancements 30
 - 16.2 Medium-Term Enhancements 30
 - 16.3 Long-Term Vision 30
- 17 Technology Stack Summary** **32**

1. System Overview

Automotive QA Intelligence is a fully offline, enterprise-grade quality analytics and diagnostics platform designed for automotive Field Technical Information Report (FTIR) analysis. The system enables technicians and quality engineers to perform natural language queries against a structured database of historical vehicle failure records, leveraging a hybrid Retrieval-Augmented Generation (RAG) architecture.

1.1 Core Objectives

- **Offline-First Architecture:** Operates entirely without internet connectivity using local LLM inference (Phi-3 Mini 4K via `llama.cpp`) and local FAISS vector indexing.
- **Hybrid Retrieval:** Combines SQL keyword search, FAISS cosine similarity vector search, and NLP entity extraction for precise multi-strategy information retrieval.
- **Intelligent Query Routing:** An NLP pipeline classifies user intent (SEARCH, ANALYTICS, VISUALIZE, COMPARE, REPORT) and dispatches queries to the optimal processing back-end.
- **Dynamic Analytics:** LLM-generated SQL queries enable ad-hoc analytics without pre-defined report templates.
- **Automated Report Generation:** Monthly PDF and DOCX quality reports are compiled dynamically from database aggregations.
- **Enterprise UI:** A Streamlit-based web interface with multi-session chat, real-time data visualization (Plotly), and citation-backed responses.

★ Key Differentiator

Unlike cloud-based AI platforms, this system operates **entirely offline** with no API keys, no external network calls, and no data egress — making it suitable for air-gapped automotive manufacturing environments where data confidentiality is paramount.

1.2 System Specifications

Table 1: Core System Specifications

Parameter	Specification
LLM Engine	Microsoft Phi-3 Mini 4K Instruct (Q4 GGUF, 3.8B parameters)
Embedding Model	all-MiniLM-L6-v2 (SentenceTransformers, 384-dimensional)
Vector Index	FAISS IndexFlatIP with IDMap (Cosine Similarity)
Database	SQLite 3 with WAL mode + 5 Materialized Views
Frontend	Streamlit 1.x with custom enterprise CSS theme
Visualization	Plotly Express + Plotly Graph Objects (6 chart types)
LLM Runtime	llama-cpp-python with NVIDIA CUDA GPU offloading
Report Formats	PDF (ReportLab) + DOCX (python-docx)
Deployment	Single-node local / on-premise (Windows/Linux)

2. End-to-End Architecture

The system follows a **layered architecture** pattern with clear separation of concerns across six layers: Presentation, Application Logic, Intelligence (AI/ML), Data Access, Storage, and Infrastructure.

2.1 Layered Architecture Diagram

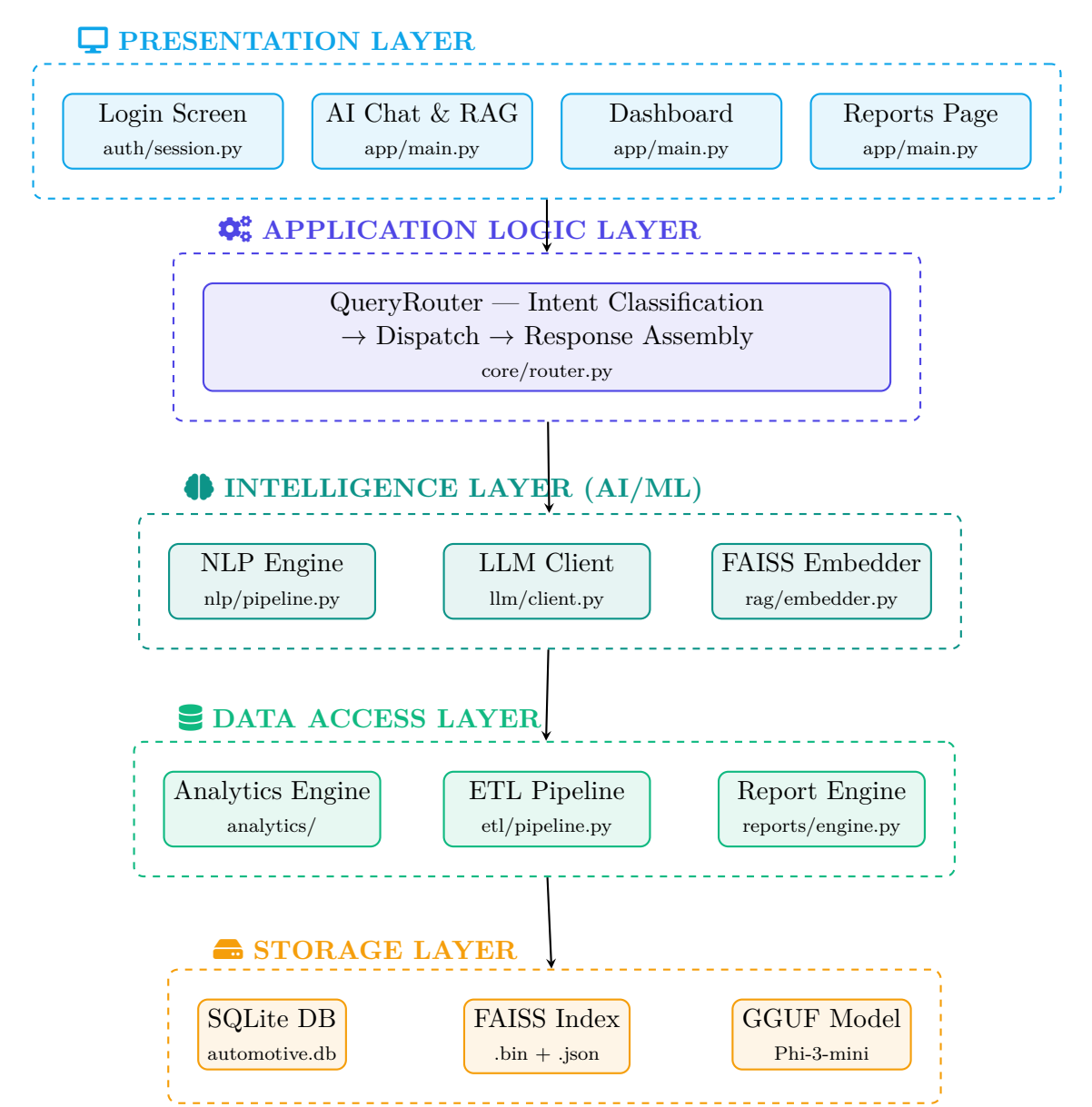


Figure 1: Five-Layer System Architecture

2.2 Module Directory Structure

Listing 1: Project Directory Layout

```
automotive_qa/
|-- app/                                # Streamlit frontend (main.py - 1103
    lines)
|-- auth/                              # User session management (session.py)
|-- core/                              # Central infrastructure
|   |-- router.py                      # Query routing & dispatch engine
|   |-- singletons.py                 # Cached resource loaders (DB, LLM,
    FAISS)
|   |-- cache.py                      # L1 (RAM) + L2 (SQLite) query cache
|   +-- paths.py                      # Absolute path resolver
|-- etl/                                # Data ingestion & transformation
|   |-- pipeline.py                   # Excel -> SQLite ETL with
    deduplication
|   +-- manual_ingest.py              # Inbox scanner + FAISS incremental
    update
|-- nlp/                                # Natural language processing
|   +-- pipeline.py                   # Intent classifier + entity
    extractor
|-- rag/                                # Retrieval-Augmented Generation
|   |-- embedder.py                   # FAISS index manager + vector
    search
|   +-- document_builder.py           # Structured text builder for
    embeddings
|-- llm/                                # Local LLM inference
|   +-- client.py                     # Phi-3 GGUF streaming client (llama
    .cpp)
|-- analytics/                          # SQL analytics engine
|   |-- engine.py                     # 12+ parameterized analytics
    queries
|   |-- graph_selector.py             # Dynamic chart type inference
|   +-- views.py                      # Materialized view refresh logic
|-- viz/                                # Plotly visualization library
|   +-- charts.py                     # 6 chart types with premium theming
|-- reports/                            # Document generation
|   +-- engine.py                     # PDF + DOCX report builder (654
    lines)
|-- data/                              # Persistent data store
|   |-- automotive.db                 # SQLite database (~6.7 MB)
|   |-- faiss_index.bin               # FAISS vector index (~9.5 MB)
|   |-- vector_metadata.json          # Index metadata
|   +-- inbox/                        # Manual ingestion drop folder
|-- models/                             # GGUF model weights
|   +-- Phi-3-mini-4k-instruct-q4.gguf (~2.4 GB)
|-- schema.sql                         # Database DDL (tables + indexes)
|-- setup.py                           # Automated project bootstrapper
+-- requirements.txt                   # Python dependencies
```

3. Data Flow & Processing Pipeline

3.1 ETL Ingestion Pipeline

Data enters the system through Excel (.xlsx) files containing FTIR records from field operations. The ETL pipeline performs column normalization, data cleaning, deduplication via MD5 hashing, computed field generation, heuristic/LLM-based summarization, and materialized view refresh — all within a single atomic transaction.

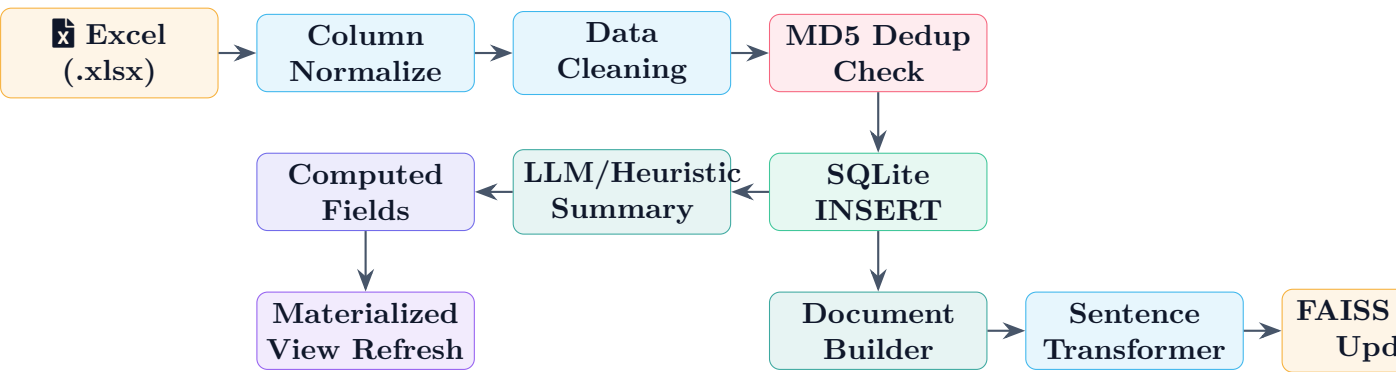


Figure 2: ETL Ingestion Pipeline Data Flow

3.2 ETL Processing Steps

Table 2: Detailed ETL Processing Steps

#	Module	Operation	Technical Details
1	etl/pipeline.py	Read Excel	pd.read_excel() with column rename normalization map
2	etl/pipeline.py	Clean & Parse	Date parsing (4 formats: DD-MM-YYYY, YYYY-MM-DD, etc.), mileage integer extraction via regex
3	etl/pipeline.py	Deduplicatio	MD5(ftir_no subject complaint) hash + UNIQUE constraint check
4	etl/pipeline.py	Computed Fields	using_km_int, report_year/month, is_resolved, has_sbpr
5	etl/pipeline.py	Summarizati	LLM 2-sentence summary or heuristic fallback from structured fields
6	analytics/view	MV Refresh	5 materialized views rebuilt in single atomic transaction
7	rag/document	Text Build	Concatenate 6 key fields into embedding document
8	rag/embedder	Vectorize	SentenceTransformer encode + FAISS IndexIDMap upsert

3.3 Query Processing Data Flow

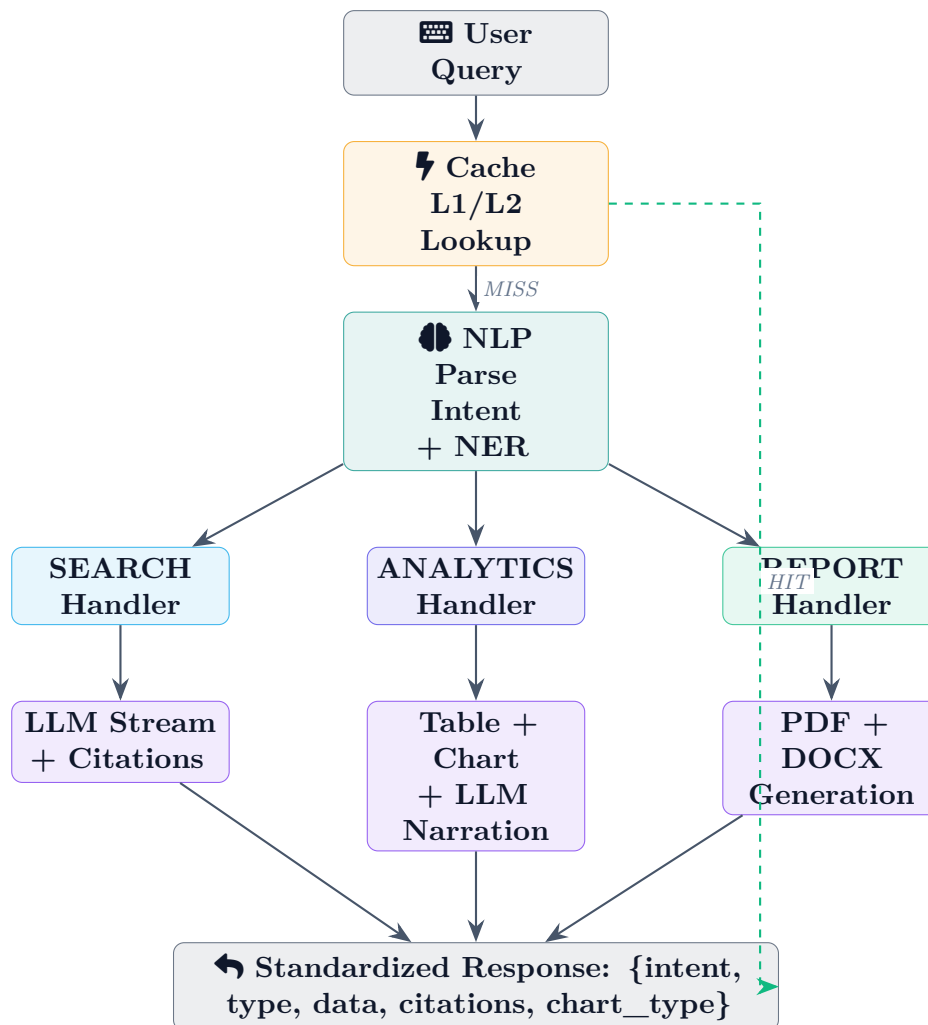


Figure 3: Query Processing Data Flow with Intent-Based Dispatch

4. Component Interactions

The following diagram illustrates the runtime interaction between all major system components during a typical user query lifecycle. Components communicate via direct function calls within a single Python process.

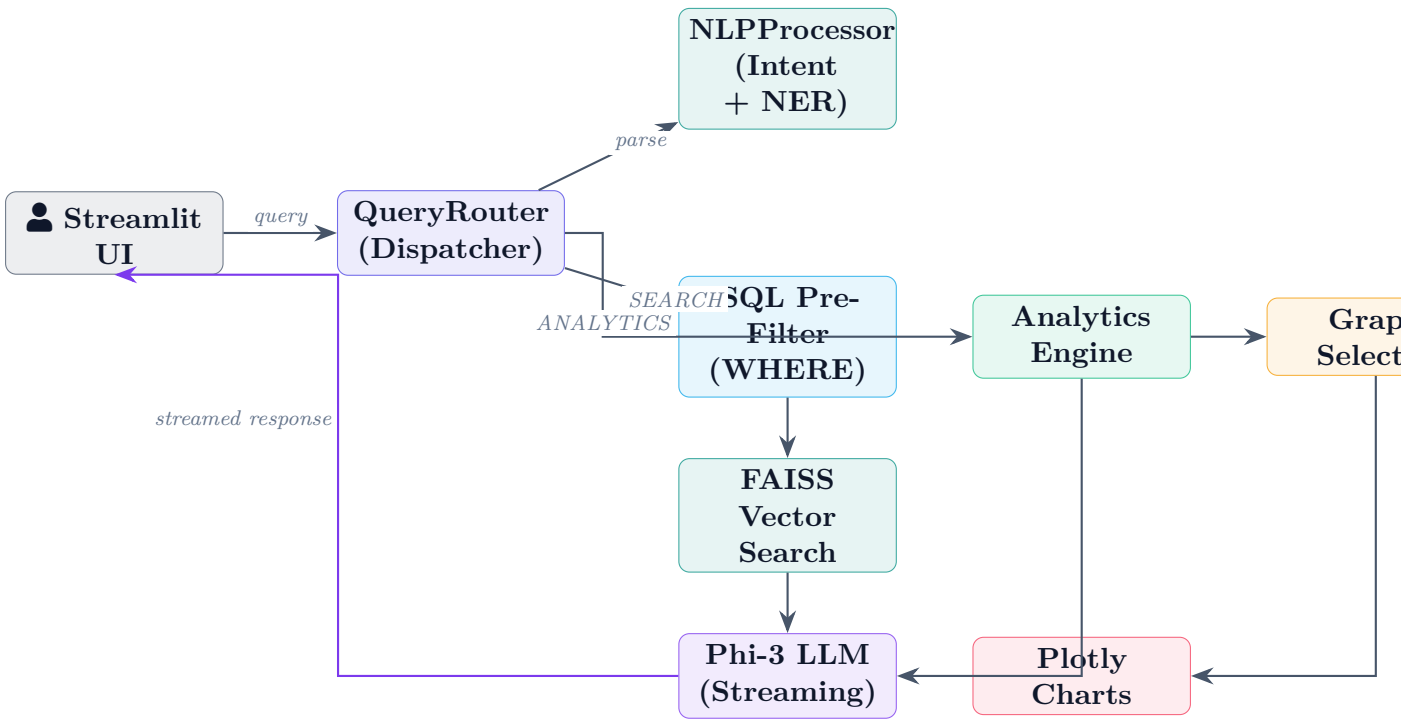


Figure 4: Runtime Component Interaction Diagram

4.1 Singleton Resource Management

All expensive resources are loaded once and cached using Streamlit’s `@st.cache_resource` decorator in `core/singleton.py`. This ensures the FAISS index (~9.5 MB), SentenceTransformer model, and Phi-3 GGUF model (~2.4 GB) are loaded into GPU VRAM/RAM only once across all Streamlit reruns.

Table 3: Singleton Resources and Their Lifecycles

Singleton	Module	Cached Resource	Lifecycle
<code>get_db_connection</code>	<code>core/singleton</code>	SQLite DB path (WAL mode verified)	App lifetime
<code>get_embedder()</code>	<code>core/singleton</code>	SentenceTransformer + FAISS index + metadata	App lifetime
<code>get_llm()</code>	<code>core/singleton</code>	Phi-3 GGUF loaded via llama-cpp (GPU offload)	App lifetime
<code>get_ingestion_tra</code>	<code>core/singleton</code>	Thread-safe ingestion status tracker	App lifetime

5. Backend Workflow

5.1 Application Startup Sequence

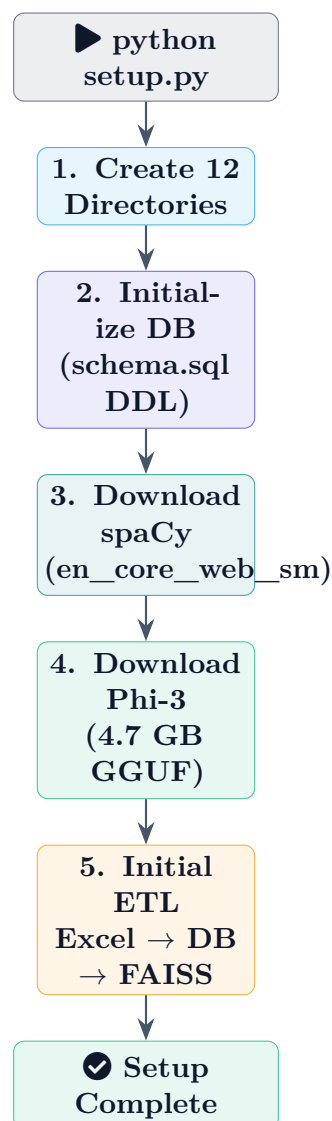


Figure 5: Project Setup Sequence

5.2 Runtime Query Processing Steps

Table 4: End-to-End Query Processing Workflow

Step	Operation	Module
1	User types query in <code>st.chat_input()</code>	<code>app/main.py</code>
2	<code>QueryRouter.dispatch_query()</code> invoked	<code>core/router.py</code>
3	QueryCache L1/L2 lookup (MD5 hash key)	<code>core/cache.py</code>
4	<code>NLPProcessor.parse_query()</code> — intent + entities + filters	<code>nlp/pipeline.py</code>
5a	SEARCH: SQL pre-filter → FAISS <code>search_subset()</code> → LLM generation	<code>core/router.py</code>
5b	ANALYTICS: LLM SQL gen (4 retries) → static fallback → GraphSelector	<code>analytics/engine.py</code>
5c	REPORT: Pass year/month params to ReportEngine	<code>reports/engine.py</code>
6	LLM streaming response via <code>st.write_stream()</code>	<code>llm/client.py</code>
7	Plotly chart rendered via <code>render_plotly_chart()</code>	<code>viz/charts.py</code>
8	Response cached to L1 RAM + L2 SQLite (2h TTL)	<code>core/cache.py</code>
9	Chat message persisted to <code>chat_history</code> table	<code>auth/session.py</code>

6. Database Architecture

The system uses **SQLite 3** with **Write-Ahead Logging (WAL)** mode for concurrent read/write access. The schema consists of **10 tables**: 1 core data table, 4 application tables, and 5 materialized view tables.

6.1 Entity Relationship Diagram

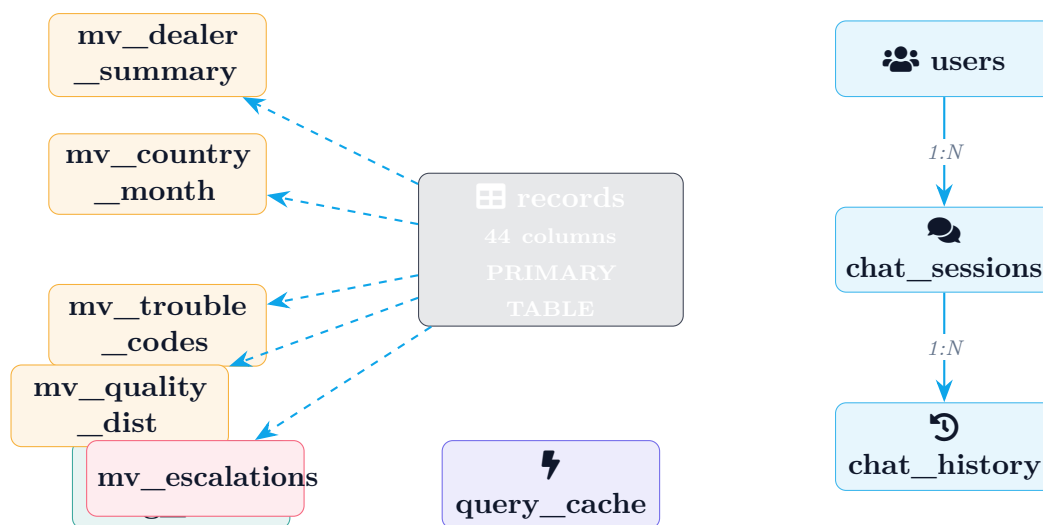


Figure 6: Database Entity Relationship Diagram

6.2 Core Schema — records Table

The **records** table stores all FTIR data with **44 columns** including 6 computed fields. It serves as the single source of truth for all analytics, search, and report generation.

Table 5: Records Table Column Groups

Group	Columns	Purpose
Identity	id, sbpr_no, ftir_no, row_hash	Primary key, deduplication
Temporal	ftir_report_date, reply_date, date_registered, date_of_incident	Date tracking
Vehicle	product_model_code, sales_model_code, vin, engine_no, transmission_no	Vehicle identification
Classification	segmentation, status, quality, fc_ok	Categorical grouping
Geography	outbreak_country, reported_company, issued_company, manufacturer_factory	Location & dealer tracking
Technical	subject, customer_complaint, trouble_code_complaint, trouble_code_defect	Failure description
Diagnostics	checked_contents, checked_results, repair_status, repair_contents	Inspection & repair
Resolution	problem_solved, action_judgement, causal_parts_no, causal_parts_name	Root cause & fix
Computed	using_km_int, report_year, report_month, is_resolved, has_sbpr, summary	Derived analytics

6.3 Materialized Views

Five materialized view tables are pre-computed from the **records** table and refreshed atomically after every ETL ingestion. These provide $O(1)$ access to common dashboard aggregations.

Table 6: Materialized View Tables

View Table	Aggregation	Refresh Trigger
mv_country_mon	Record count by country, year, and month	Post-ETL ingestion
mv_trouble_code	Frequency count by trouble code	Post-ETL ingestion
mv_dealer_summ	Record count by reporting dealer/company	Post-ETL ingestion
mv_quality_dist	Distribution count by quality rating	Post-ETL ingestion
mv_escalations	Unresolved + Poor quality cases (critical)	Post-ETL ingestion

6.4 Index Strategy

Nine B-tree indexes cover 95%+ of filter patterns used in analytics and search queries:

Table 7: Database Index Configuration

Index Name	Column	Primary Use Case
idx_outbreak_country	outbreak_country	Country-based filtering & grouping
idx_product_model_cc	product_model_cod	Model-specific analytics
idx_segmentation	segmentation	Engine/Transmission category filters
idx_trouble_code_com	trouble_code_compl	DTC frequency analysis
idx_status	status	Status-based filtering
idx_quality	quality	Quality distribution queries
idx_repair_status	repair_status	Repair status analytics
idx_using_km_int	using_km_int	Mileage range queries
idx_report_year	report_year	Temporal analytics

7. API & Query Flow

The system does not expose a REST API — all interactions occur through the Streamlit UI. However, the `QueryRouter` acts as an **internal API gateway** that routes classified intents to the appropriate processing backend.

7.1 Internal Query API Contract

Table 8: QueryRouter Response Dictionary Schema

Field	Type	Description
intent	str	Classified intent: SEARCH, ANALYTICS, VISUALIZE, COMPARE, REPORT, AMBIGUOUS
type	str	Response type: text_stream, table_stream, table_only, report
data	mixed	Messages list for LLM, or DataFrame, or report params
citations	list[dict]	Referenced FTIR records with ftir_no, subject, company, country
chart_type	str	Plotly chart: horizontal_bar, line, donut, histogram, radar, grouped_bar
chart_title	str	Generated chart title string
sql_query	str	The actual SQL query executed (for transparency)

7.2 Intent Classification Rules

Table 9: NLP Intent Classification Scoring System

Intent	Score	Trigger Keywords	Handler
VISUALIZE +EXPLAIN	4	(chart graph plot) AND (why explain cause)	<code>_handle_analytics()</code>
VISUALIZE	3	chart, graph, plot, visualize	<code>_handle_analytics()</code>
COMPARE	3	compare, vs, versus, difference between	<code>_handle_analytics()</code>
REPORT	3–6	monthly report, generate report, export + year	Report params
ANALYTICS	2	top, count, how many, total, frequency, average	<code>_handle_analytics()</code>
SEARCH	3	similar, find, cases like, search, lookup	<code>_handle_search()</code>
AMBIGUOUS	0	No keyword match (fallback to search)	<code>_handle_search()</code>

8. AI / ML Pipeline

8.1 NLP Processing Pipeline

The NLP pipeline in `nlp/pipeline.py` performs three sequential tasks: intent classification, named entity recognition (NER), and filter extraction. It uses a dual-mode approach: **spaCy** with custom **EntityRuler** patterns as primary, with **regex-based fallback**.

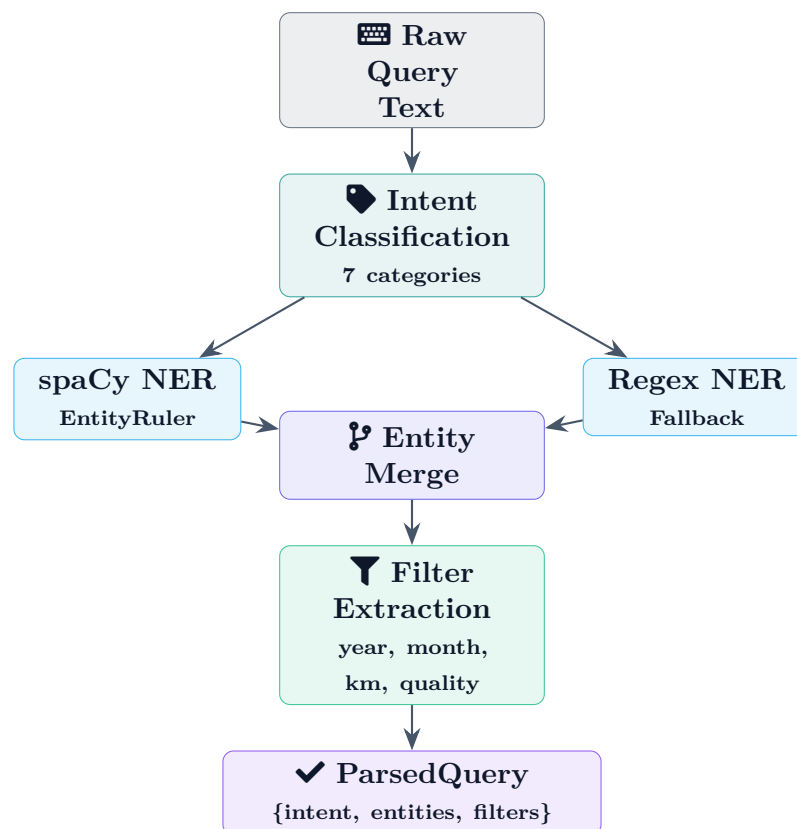


Figure 7: NLP Processing Pipeline

8.2 Named Entity Recognition

Table 10: Entity Types and Recognition Methods

Entity Type	Recognition Method	Pattern Example	DB Validation
TROUBLE_CODE	Regex: [PBCU]\d{4}	P0500, C0035	No (format-based)
PRODUCT_MODEL	Regex: Y[A-Z0-9]{5,9}	YNC412, YFBP51	Yes (cross-checked)
COUNTRY	Lookup list from DB	India, Brunei	Yes (loaded at init)
SALES_MODEL	Regex: [A-Z]{3}\d{3,4}	ERT701, ATM412	No (format-based)
VIN	Regex: MA3[A-Z0-9]{14}	MA3FJEB1S00	No (format-based)
FTIR_NO	Regex: FTIR/\d{4}/\d{4}	FTIR/2024/101	No (format-based)

8.3 Local LLM Inference Engine

The LLM subsystem uses **Microsoft Phi-3 Mini 4K Instruct** (3.8B parameters, Q4 quantized GGUF format) running locally via `llama-cpp-python` with full CUDA GPU layer offloading.

Table 11: LLM Operational Modes

Role	Function	Temp	Max Tok	Stop Tokens
Chat Response	<code>generate_chat_st</code>	0.1	300	<code>< end ></code> , <code>< user ></code>
Record Summary	<code>generate_summar</code>	0.1	150	<code>\n, Summary:</code>
SQL Generation	<code>query_via_llm()</code>	0.1	500	<code>< end ></code> , <code>< user ></code>

8.4 LLM-Generated SQL with Self-Healing

The `AnalyticsEngine` implements a sophisticated LLM-to-SQL pipeline with automatic error recovery. When the LLM generates an invalid SQL query, the error message is appended to the prompt context, and the LLM is asked to self-correct — up to **4 retry attempts**.

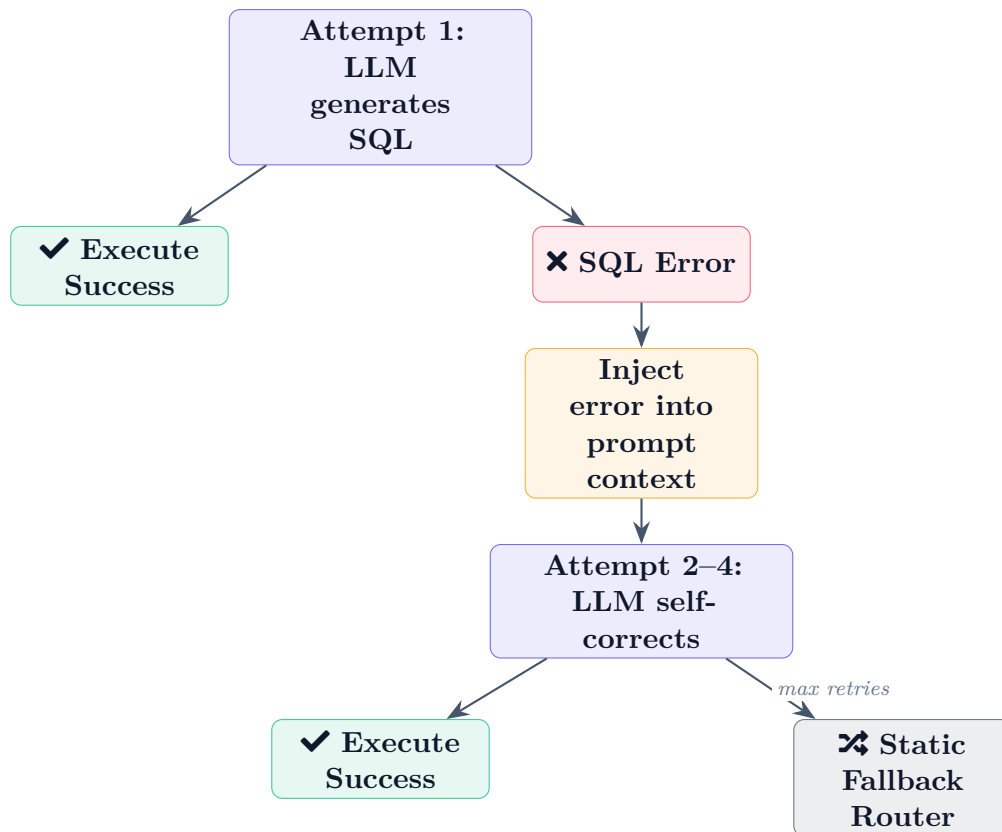


Figure 8: LLM SQL Self-Healing Pipeline (4-Retry Mechanism)

9. Retrieval Pipeline (SQL + FAISS + RAG)

The SEARCH intent triggers a **three-stage hybrid retrieval pipeline** that combines structured SQL filtering, FAISS vector similarity search, and LLM-powered response generation with source citations.

9.1 Three-Stage Hybrid Retrieval

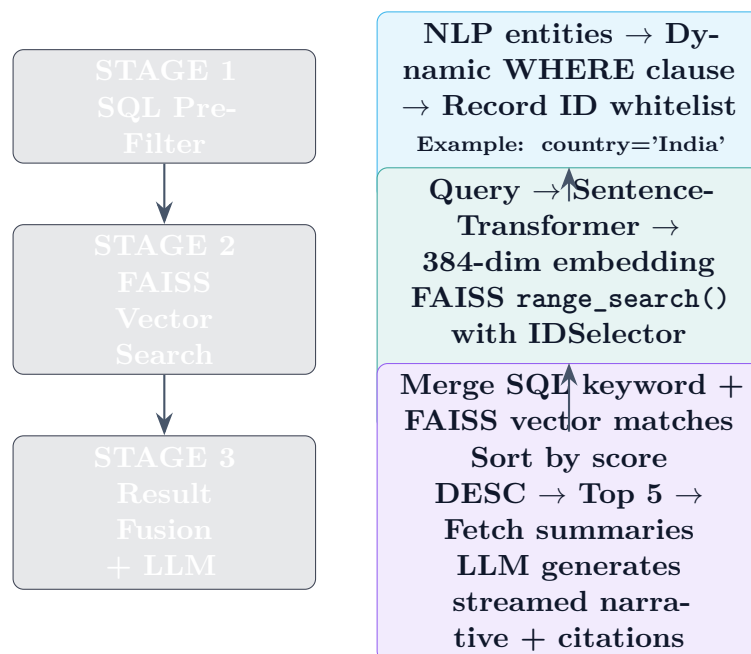


Figure 9: Three-Stage Hybrid Retrieval Architecture

9.2 Vector Index Architecture

Table 12: FAISS Vector Index Specifications

Parameter	Value	Rationale
Index Type	IndexFlatIP + IndexIDMap	Exact inner product search with custom record IDs
Embedding Model	all-MiniLM-L6-v2	384-dim, fast inference, strong semantic similarity
Similarity Metric	Cosine (via normalized IP)	L2 normalized vectors $\Rightarrow$ IP = Cosine
Document Fields	6 concatenated text fields	subject, complaint, checked, results, repair, part
Metadata	id, country, model, code, segment	Enables pre-filter ID selection
Search Method	<code>range_search()</code> + <code>IDSelectorArray</code>	Threshold-based filtering on whitelisted subset
Persistence	<code>faiss_index.bin</code> + <code>vector_metadata.json</code>	Binary index + JSON metadata on disk

9.3 Embedding Cache Strategy

To avoid redundant SentenceTransformer inference, all computed embeddings are cached in the `embedding_cache` SQLite table as BLOB-serialized NumPy arrays. The cache key is an MD5 hash of the input text, providing $O(1)$ lookup. This is critical during ETL batch ingestion where thousands of documents are processed.

9.4 SQL Keyword Search (BM25-like)

Alongside vector search, the system performs keyword-based SQL LIKE queries across four text columns: `subject`, `customer_complaint`, `causal_parts_name`, and `summary`. Keywords are extracted by the NLP pipeline (nouns, proper nouns, adjectives) with stop-word filtering. Results are intersected with metadata-whitelisted IDs and merged with FAISS scores (baseline score = 0.35 for SQL-only matches).

10. Frontend Architecture

The frontend is built with **Streamlit** and features a multi-page layout with three main views, custom enterprise CSS theming (Inter font family, glassmorphism cards), and interactive chat with streaming LLM responses.

10.1 Page Structure

Table 13: Frontend Page Structure

Page	Route	Features
AI Chat & RAG	 AI Chat & RAG	Chat interface, streamed LLM, citation cards, SQL viewer, Plotly charts, PDF export
Quality Dashboard	 Quality Dashboard	4 KPI metric cards, trouble code chart, trend line, quality donut, model comparison
Monthly Reports	 Monthly Reports	Year/month selector, PDF + DOCX generation, download buttons

10.2 UI Component Hierarchy

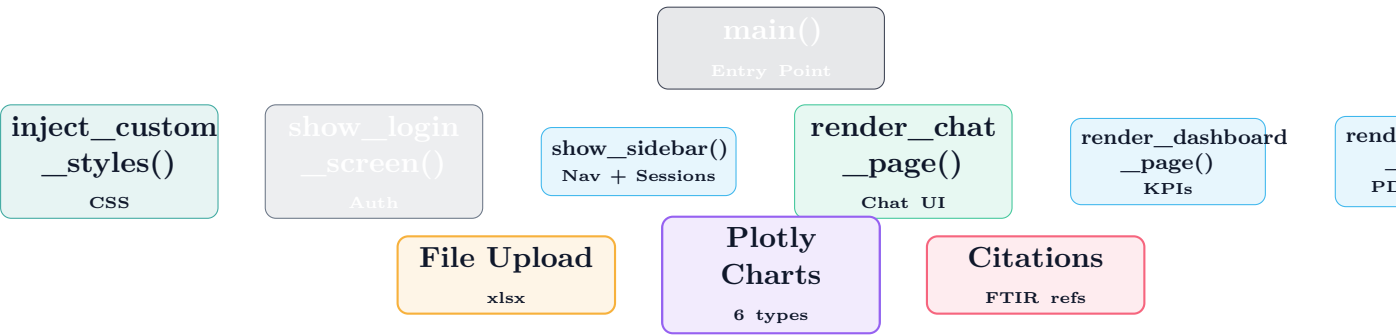


Figure 10: Frontend Component Hierarchy Tree

10.3 Visualization Chart Types

Table 14: Dynamic Chart Type Selection (GraphSelector)

Chart Type	Function	Use Case	Auto-Selected When
horizontal_bar	plot_horizontal_bar	Rankings	failures/count column detected
line	plot_line_trend()	Temporal trends	period/report_year column
donut	plot_donut_chart	Distribution	quality column + 2 cols only
histogram	plot_histogram()	Mileage dist.	mileage column detected
radar	plot_radar_comparison	Multi-metric	COMPARE + resolution_rate
grouped_bar	plot_grouped_bar	Comparison	3+ numeric columns

11. Deployment Architecture

The system is designed for **single-node on-premise deployment** on Windows or Linux workstations with NVIDIA GPU acceleration. No cloud services, API keys, or network connectivity are required.

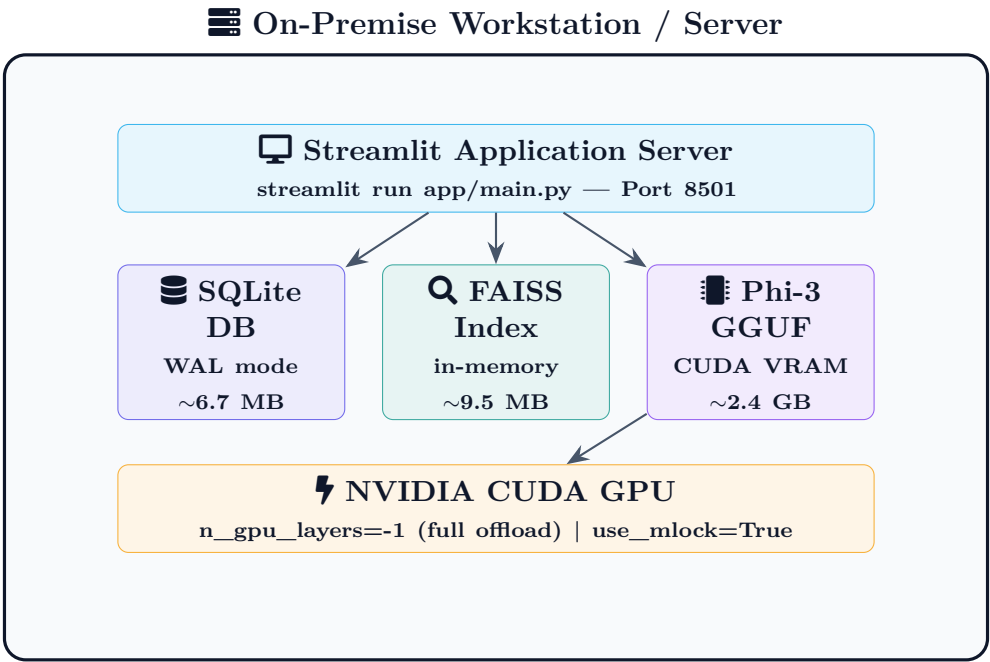


Figure 11: Deployment Topology

11.1 System Requirements

Table 15: Hardware & Software Requirements

Resource	Minimum	Recommended
RAM	8 GB	16+ GB
GPU VRAM	4 GB (NVIDIA CUDA)	8+ GB
Disk Space	10 GB	20+ GB (models + data growth)
Python	3.9+	3.10+
CUDA	11.x+	12.x
Toolkit		
OS	Windows 10+ / Linux	Windows 11 / Ubuntu 22.04

11.2 Startup Commands

Listing 2: Application Launch Commands

```
# Initial setup (one-time):
```



```
cd automotive_qa
pip install -r requirements.txt
python setup.py

# Launch application:
streamlit run app/main.py
```

12. Scalability Considerations

Table 16: Scalability Assessment & Mitigation Strategies

Dimension	Current Design	Scaling Strategy	Threshold
Record Count	SQLite single-file	Migrate to PostgreSQL with partitioning	> 500K records
Vector Index	FAISS IndexFlatIP	Switch to FAISS IVF-PQ or HNSW	> 1M vectors
LLM Inference	Single Phi-3 on 1 GPU	Model parallelism, vLLM, or batching	> 10 concurrent
Concurrent Users	Streamlit single-threaded	NGINX + multiple Streamlit workers	> 5 concurrent
ETL Volume	Row-by-row insertion	Batch INSERT with <code>executemany()</code>	> 10K rows/file
Cache Size	200-entry RAM LRU	Redis with configurable TTL	> 1000 queries/-day
Embedding	CPU + GPU inference	Pre-computed embeddings, batch encoding	> 50K documents

13. Performance Optimizations

The system implements **10 key performance optimizations** spanning caching, indexing, model management, and query execution.

Table 17: Performance Optimization Matrix

Optimization	Implementation	Impact
Two-Tier Query Cache	L1 RAM (dict, 200 entries) + L2 SQLite (TTL 2 hours)	Eliminates duplicate LLM calls
Embedding Cache	SQLite BLOB storage of NumPy arrays (MD5 key)	Avoids re-encoding in ETL
Singleton Resources	<code>@st.cache_resource</code> for DB, LLM, FAISS	One-time model loading
WAL Journal Mode	<code>PRAGMA journal_mode=WAL</code>	Concurrent reads during writes
FAISS IDSelector	Pre-filter search with <code>IDSelectorArray</code>	Reduces vector comparisons
Dynamic Threshold	0.30 (SQL match) vs 0.45 (no match)	Reduces false positives
Table Truncation	<code>df.head(20) + describe()</code> for large DFs	Prevents context overflow
9 B-Tree Indexes	95% filter pattern coverage	Sub-ms SQL execution
GPU Full Offload	<code>n_gpu_layers=-1 + use_mlock=True</code>	Max LLM throughput
Materialized Views	5 pre-aggregated tables	$O(1)$ dashboard queries

 **Caching Architecture Detail**

The two-tier cache operates as follows:

- L1 (RAM):** An in-memory Python dictionary keyed by MD5 hash of the query string. Maximum 200 entries with automatic FIFO eviction and expiration-based pruning.
- L2 (SQLite):** Persistent `query_cache` table storing JSON-serialized results with configurable TTL (default 2 hours). Supports both user-scoped and globally shared cache entries.

On cache *hit*, the full router response (including DataFrames, chart types, and citations) is returned instantly without invoking the LLM or FAISS subsystems.

14. Security Considerations

Table 18: Security Assessment Matrix

Domain	Implementation	Assessment
Authentication	Passwordless username-based login (verify_or_create_user)	Trusted internal only
Data Isolation	User-scoped chat sessions (user_id FK)	Own history only
Data at Rest	SQLite with OS-level file permissions	No encryption
Data in Transit	Localhost only (no network exposure)	No TLS needed
AI Privacy	Fully offline LLM — no external API calls	No data egress risk
SQL Injection	Parameterized queries (?) throughout	Fully protected
File Upload	Restricted to .xlsx only via Streamlit uploader	Type validation
LLM Safety	Temperature 0.1 + strict stop tokens	Minimal hallucination
Sessions	Streamlit session_state (per browser tab)	Cleared on logout

 **Security Advisory**

The current authentication is **passwordless** and suitable only for trusted internal networks. For production deployment exposed to wider networks, implement:

- Password hashing via **bcrypt**
- JWT-based session tokens
- Role-based access control (RBAC): admin, analyst, viewer
- HTTPS termination via reverse proxy (NGINX + TLS certificates)

15. Error Handling & Resilience

The system implements comprehensive error handling with **graceful degradation** at every layer:

Table 19: Error Handling Strategy Matrix

Error Scenario	Handling Strategy	Module
LLM SQL generation failure	4-retry self-healing with error context injection	analytics/engine
LLM model file not found	<code>FileNotFoundException</code> with descriptive message	llm/client
spaCy model not installed	Fallback to regex-only NER extraction	nlp/pipeline
FAISS index missing	Automatic rebuild from database records	rag/embedder
Duplicate FTIR records	MD5 hash + UNIQUE constraint → skip silently	etl/pipeline
Excel column name mismatch	Column rename normalization map	etl/pipeline
Date parsing failure	Try 4 formats; fallback to raw string	etl/pipeline
LLM summary failure	Heuristic 2-sentence summary fallback	etl/pipeline
Unrelated user query	FAISS threshold filter → general assistant fallback	core/router
Empty analytics result	<code>text_stream</code> : “No data available”	core/router
Chat session deletion	CASCADE delete of chat_history records	auth/session
Query cache expiry	Auto-delete from SQLite on next access	core/cache
MV refresh failure	ROLLBACK transaction; log error	analytics/views
CUDA DLL not found	Dynamic DLL directory registration at startup	core/singletons

16. Future Improvements

16.1 Short-Term Enhancements

- **Multi-Turn Conversation Context:** Pass previous chat messages to the LLM for context-aware follow-up answers.
- **Advanced Authentication:** Add password hashing (bcrypt), role-based access control, and JWT session tokens.
- **Chart Embedding in Reports:** Render Plotly charts as PNG images and embed them in monthly PDF reports.
- **Real-Time ETL Watchdog:** Use filesystem event monitoring (watchdog library) for automatic inbox ingestion.
- **Batch ETL Optimization:** Replace row-by-row INSERT with `executemany()` for 10×+ ingestion performance.

16.2 Medium-Term Enhancements

- **PostgreSQL Migration:** Replace SQLite with PostgreSQL for true concurrent multi-user access and table partitioning.
- **FAISS IVF-PQ Index:** Upgrade to Inverted File with Product Quantization for sub-linear search at 1M+ scale.
- **Larger LLM Models:** Support Qwen 2.5 7B (already downloaded) or Llama 3.1 for improved SQL generation.
- **REST API Layer:** Expose FastAPI endpoints for headless integration with other enterprise systems.
- **Automated Testing:** Add `pytest` suite covering NLP classification, ETL pipeline, and analytics correctness.

16.3 Long-Term Vision

- **Kubernetes Deployment:** Containerize (Docker) and deploy on K8s with horizontal pod autoscaling.
- **Real-Time Streaming:** Integrate with Kafka or MQTT for live field data from IoT-connected vehicles.
- **Predictive Analytics:** Train time-series models (Prophet/ARIMA) on failure trends for proactive alerts.
- **Knowledge Graph:** Build a Neo4j knowledge graph linking models, parts, failure modes, and repairs.

- **Multi-Language Support:** Extend NLP pipeline for Japanese, Korean, and other OEM-relevant languages.

17. Technology Stack Summary

Table 20: Complete Technology Stack

Category	Technology	Version	Role
Language	Python	3.9+	Primary development language
Web Framework	Streamlit	1.x	Frontend UI and application server
Database	SQLite 3	WAL mode	Structured data storage
Vector Store	FAISS (faiss-cpu)	1.x	384-dim cosine similarity index
Embedding	SentenceTransfor	all-MiniLM-L6-v2	Text → 384-dim vector encoding
LLM Runtime	llama-cpp-python	CUDA build	Local GGUF model inference
LLM Model	Phi-3 Mini 4K	Q4 GGUF (2.4 GB)	Chat, summary, SQL generation
NLP	spaCy	en_core_web_sm	NER and keyword extraction
Visualization	Plotly	Express + GO	6 interactive chart types
Data Processing	Pandas	2.x	DataFrame operations
PDF Generation	ReportLab	4.x	Professional PDF rendering
DOCX Generation	python-docx	0.8+	Word document reports
Excel Parsing	openpyxl	3.x	FTIR Excel file reading
Numerical	NumPy	1.x	Vector array operations
GPU Acceleration	NVIDIA CUDA	11.x+	LLM inference GPU offloading

— End of Document —

Document generated on June 11, 2026

Automotive QA Intelligence — System Architecture Documentation v1.1